Scripts Julian

-- Written by Julian Donner -- last updated 26 October 2021 --

This page contains a summary of scripts for pulsar and DM studies that could be useful for other people. These scripts come with absolutely no warranty. Feel free to use or share these scripts as you see fit.

Note: The path **/lofardata/** used in this document contains the Bielefeld LOFAR archive. It can currently be accessed directly from lofarfs. It is also mounted in wsd4so via nfs as **/nfs/lofardata/** or (read-only) /nfs-ro/lofardata/. It is also planned to be mounted on atwood (read-only). The scripts that work directly with /lofardata/ will work with both paths.

Most of the scripts can be run with the '-h' option to get more detailed instructions on how to run these scripts.

Contents

DM Utilities

These scripts are located in /lofardata/scripts/julian/DMutil/.

DM calculation:

- **jd_calcDM.bash:** Calculate a single DM for the given set of ToAs (timfile). Can apply different ToA rejection schemes and has options to ignore ToAs outside a given frequency range. The usual use case is to calculate the DM of a single observation. I highly recommend always using the '-n' option, which disables the multiplication of the DM uncertainty by sqrt(red_chi^2) if red_chi^2 is less than 1. Requires jd_funcs.bash and jd_calc_stats.bash to be located in the same directory (see below).
- **jd_calcDMseries.bash:** Calculate a DM time series for a given set of ToAs (timfile). There are options to slice the dataset into chunks of a given length. This script uses jd_calcDM.bash (see above), which needs to be located in the same directory as this script. It can also pass arguments to jd_calcDM.bash, e.g. to apply outlier rejection or enabling the '-n' option.
  Example command: *jd_calcDMseries.bash -p=mypar -t=mytim -g=0.5 -m=3 -a="-n -O"*

DM utility:

- **jd_calc_avg.bash:** Calculate the average of y values over intervals of x. Can be used to compute an averaged DM time series (e.g. 1 DM value every X days). A possibly more useful functionality is to calculate a running average of a DM time series. The Gaussian shape appears smoother, while the exponential shape is calculated much more quickly.
  Example command: *jd_calc_avg.bash -x=1 -xr=60 -y=2 -ye=3 -r mydmfile.txt*
- **jd_addDMcorr.bash:** Add '-dm' or '-dmo' flags to the ToAs in the given timfile. Interpolate the DM between the nearest DM values in the given DM time series. (First column must be MJD). A common use case would be to apply a DM time series that was smoothed using a running average with the above script jd_calc_stats.bash.
  Example command: *jd_addDMcorr.bash -col=2 -dmo mytim.tim mydmfile.txt*
- **apply_DMX.bash:** Adds DMX parameters (all set to 0 and fitting enabled) with a given step size to a given parfile. Also requires a timfile to determine the periods of time where the DMX

parameters are required. After using this script, the next step is usually to run tempo2 and refit the timing model.
Example command: *apply_DMX.bash mypar.par mytim.tim 30 > mypar_with_DMX.par*

- **extract_DMX.bash:** Extract the DMX parameters from a given parfile as a DM time series (e.g. for plotting).

Structure function (SF):

- **jd_calc_sf.py**: Calculate the SF of a given DM time series. Can also be used to run Kolmogorov simulations and fit a Kolmogorov model amplitude to the data-derived SF.
Example command: *jd_calc_sf.py -M 1 -D 2 -U 3 -t 7 -i 1.2 -s -n 1000 mydmfile.txt*

Disentangle IISM from Solar Wind (SW):

- **jd_estimate_iism_dm_vars.py:** Take a given DM time series affected by the SW. Fit a cubic spline through the Solar oppositions and use this as an IISM model, which is stored to a file. If this model is adequate, the residuals should only contain the Solar wind. This can also be tested with this script, as it can fit for the SW amplitude for each year and overplot the theoretical maximum/minimum (i.e. only slow/fast wind, see You et al, 2007 or Tiburzi et al, 2019). It is also possible to print the DM time series with the annually-fitted SW model subtracted.
While this script may not be adequate to fully disentangle the IISM from the SW, it is certainly good enough to study the SF of the IISM-induced DM variations. See also Section 5.2.3 of my PhD thesis.

Disentangle scattering and DM:

- **jd_getDM_model_scattering_B.py:** Take an analytic profile model and fit its the width of its scattering tail to each profile in the input archives. Then fit a dispersive slope to the phases of the individual frequency channels to obtain a DM. See also Section 6.2.4, Approach B, of my PhD thesis.
- **jd_getDM_model_scattering_C.py:** This might be of limited use. Fit a dispersive slope as well as a frequency-dependent scattering tail that follows a power law simultaneously to the data profiles. The intrinsic profile assumed to be Gaussian in shape. See also Section 6.2.4, Approach C, of my PhD thesis.

Used by other scripts. These should always be located in the same directory as the scripts that use them:

- **jd_calc_stats.bash:** Can be used to calculate a mean/median/rms/chi^2 of some column in some datafile (can also use stdin). A constant can also be subtracted from each row of the datafile first, e.g. to calculate an RMS deviation. This script is used by other scripts, but can however also be of use on its own.
- **jd_funcs.bash:** Contains functions used by other scripts.

General Pulsar Utilities

These scripts are located in /lofardata/scripts/julian/PSRutil/.

- **reset_wt.py:** Resets the weights in a given archive, i.e. set the weight of all profiles to 1. This can be used to undo RFI removal.
- **set_profile_wt.py:** Written by Stefan Osłowski. Set the profile weights in an observation, e.g. depending on S/N ratio.
Example command: *set_profile_wt.py -R --snrsq myfile.ar*
- **getRFIfraction.bash:** Print the average RFI fraction of all given TIMER/PSRFITS files. Can also print the RFI fraction of each file individually. The RFI fraction is estimated from profile weights in case of scrunched archives, and the result can be a bit off in that case (and will be far off if

the weights were modified). It is therefore safer to use unscrunched archives.
Another way to see the RFI fraction of an observation accurately is to get it from the pre-processing log file, e.g.: *grep RFI_FRAC /lofardata/processed/DE601/J0700+6418/2013-07-25-10\:28/freq_append.log*

- **shape_core_archive.bash:** A "dirty" way of making a LOFAR core observation (400 channels) match a common GLOW observation (366 channels), where the top 6 channels were removed to allow for more different scrunching options. Takes a single TIMER/PSRFITS file with 400 channels as argument and will produce a new file with 360 channels. The reason why this script is "dirty" is that it needs to add 2 empty dummy frequency channels at the top of the band.
- **jd_waterfall.py:** Plot a waterfall of the profiles of the given archive(s). The script can also apply scrunching to the input files. Another option is to plot a waterfall of profile differences to a reference profile.
- **jd_sky_movement.py:** Plot the apparent motion of the pulsar in the sky. Can also be used to plot a DM time series on top of the pulsar's path using a color scale.
- **jd_get_solarangle.py:** Calculate the angular distance for a given (pulsar) position (in ecliptic coordinates, specified in degrees) using equations gathered from the internet.
- **jd_get_solarangle_astropy.py:** Same as above, but use astropy to calculate the angular distance. This is probably more reliable, but I cannot remember if it is. Also requires the astropy package (obviously).
- **convert_ecliptic_to_equatorial.py:** Convert the given parfile from ecliptic to equatorial coordinates, including proper motion, using astropy.
- **fscrunch_cut.py:** Cut away any frequency channel from an archive that is not present in a given reference archive. Can be used to apply f-scrunching afterwards.

GLOW/LOFAR transfer and pre-processing

(Paths are noted here as /lofardata/..., but if mounted via an nfs (currently on wsd4so), the path /nfs/lofardata/... will also be accepted by the script.)

There are 3 databases:

1. /lofardata/<site>/<psrj>/<date>/: "raw" pulsar observations in TIMER or PSRFITS (since 2020?) as they come out of the observing system. The observation stay here as long as they were not successfully pre-processed.
2. /lofardata/processed/<site>/<psrj>/<date>/: pre-processed observations (cleaned, calibrated, updated ephemeris, t-scrunched to 60-s-subints for most sources).
3. /lofardata/trashbin/<site>/<psrj>/<date>/: the "raw" data files of observations that were already pre-processed and may be deleted if space is needed.

Downloading GLOW data:

- **/lofardata/scripts/julian/rsync_from_juelich.bash:** This script can be used to download the regular GLOW observations from Jülich to /lofardata/ and remove them from the nfs in Jülich using rsync. So this script completes step 9 in the metadata procedure: https://www.glowconsortium.de/index.php/en/internal/2016-02-18-09-13-27/technical-notes/168-metadata (9. Copy data to Bielefeld (lofarfs) and remove from Jülich.)
For this script to work, you need write permissions in /lofardata/ and password-free login to observer@glow.fz-juelich.de, i.e. your public ssh key needs to be in the list of authorised keys in Jülich.

Downloading LOFAR core data:

- Login to the LTA: https://lta.lofar.eu/
- Go to advanced search --> all observations and pipelines

- Put in start and end date (only a few month, because if we try to stage more at once, it will just throw an error)
- Set max number of obs per page to 1000
- Select all observations
- Go to show dataproducts
- Select PulpSummaryDataProduct
- Stage
- When the staging finished, download the staged files. This can probably be done in multiple ways. One is to use wget: wget -i html.txt (you might need to setup your login credentials with user=<string> and password=<string> in your ~/.wgetrc for this to work)
- This will leave you with a bunch of *summary*.tar files.
- **/lofardata/scripts/julian/extract_core.bash:** Run this script in the folder where all the *summary*.tar files from the LTA are located. This script will extract the relevant files from the .tar files and sort them into /lofardata/.

Pre-processing (/lofardata/scripts/julian/preprocessing/).

- **iterative_cleaner.py:** Iterative RFI cleaner written by Lars Künkel, based on the Surgical algorithm of the coastguard python package. (see https://github.com/larskuenkel/iterative_cleaner)
- **freq_append.py:** This script is the core of the GLOW pre-processing. It appends the lanes (frequency bands) of an observation, cutting away subints in individual lanes if needed (more advanced than psradd). The script can also run RFI cleaning (requires iterative_cleaner.py, e.g. in the same directory as freq_append.py), polarisation calibration using dreamBeam (https://github.com/2baOrNot2ba/dreamBeam, has to be properly installed), and basic functions like updating the ephemeris and scrunching. Run freq_append.py -h for more detailed information.
- **process.bash:** This is the script to run for the pre-processing. It runs freq_append.py (see above) and handles the directory tree and permissions. The user is required to have write permissions in /lofardata/. The script goes through all telescope sites it knows (currently DE60X, FR606, SE607, LOFAR) in /lofardata/<site> and searches for pulsar observations. It will run the pre-processing and write the results to /lofardata/processed/<site>. If that runs smoothly, the original observation from /lofardata/<site> are moved to the trashbin /lofardata/trashbin/<site>. The script accepts either the '-a' option to process all pulsars or '-p=LIST' to process a single pulsar or a list of pulsars. If more than 2% of a lane is cut away by freq_append.py (usually due to a lane failed full recording), processed versions of each lane are also kept individually.
  By default, the script ignores any observation for which the processing was already started before (i.e. the logfile in /lofardata/processed/<site>/<psrj>/<date>/ exists). Therefore, the script can be run in multiple instances in parallel (e.g. in multiple tabs of a screen). 4 instances on a machine with 64 GB of RAM are usually no problem. A single process uses only a single CPU core, so RAM is more likely to be the limiting factor. In case freq_append.py failed for some reason unrelated to the data quality, the script can be forced to re-try observations that were already tried by using the -r option. This option should *not* be used in parallel on the same pulsar, as then the parallel scripts would try to process the same observation. See the help (process.bash -h) for more information.
- **parfiles/:** This directory contains all parfiles of pulsars that can be processed. Add a parfile of a new pulsar here or update existing parfiles. These parfiles will always be installed to the processed observations before scrunching.
- **pulsars.dat:** This file contains settings for the processing. For each source, the tsub (length of a single subintegration) can be specified as well as the maximum iterations in cleaning the RFI (more iterations are suggested for scintillating sources). There is also a DEFAULT line specifying default options used for most pulsars. This file should not be added frequently (except in cases when new pulsars are added, though it is usually not required) to avoid a highly inhomogeneous dataset.

Trashbin (/lofardata/trashbin/):

- **delete.bash:** Deletes data from the trashbin to free up space in /lofardata. For each observation, this script double-checks whether there is actually a processed version of this observation. The script is not executable by itself (just another layer of safety). Run it with "bash delete.bash"
- **to_delete:** This file contains a list of pulsars for which data are currently deleted. The delete.bash script will only touch data from these pulsars. While in principle there should be no need for any of the data in the trashbin, it is always a good idea to only delete data if it is absolutely necessary. Therefore the trashbin is only emptied as much as needed.
- **trashbin_stats.bash:** Displays disk usage per pulsar. Useful to determine which pulsars to add to the file to_delete. This script also prints how much disk space would currently be freed by running delete.bash.

LOFAR Results

The standard templates, ToAs, parfiles and DM time series derived in my thesis (see Chapter 7) can be found in: /lofardata/scripts/julian/results_thesis/

All the data are based on data containing 360 frequency channels, i.e. a bandwidth of 70.3 MHz centred at 153.2 MHz. For each pulsar, the following files are given:

PSRJ_LOFAR_Ychan.std	standard template, Y = number of frequency channels in the template (always equals 1 or X, see below)
PSRJ_LOFAR_Xchan.tim	ToAs, X = number of frequency channels used per observation
PSRJ_LOFAR_DM.txt	DM fits for each observation, including outlier rejection
PSRJ_LOFAR_DM_running_average.txt	running average of the DM time series
PSRJ_LOFAR_Xchan_outliers_rejected.tim	ToAs, all outliers removed (as determined during the DM fits)
PSRJ_LOFAR_Xchan_DMcorrected.tim	ToAs (outliers rejected) with -dmo flags for DM corrections (using PSRJ_LOFAR_Xchan_DM_running_average.txt)
PSRJ_LOFAR.par	final timing model (not modelling timing noise)